SOLYTI

Powershell

MAHOUTO
MICHAELIS



Introduction

- PowerShell est une solution **multiplateforme d'automatisation des tâches**, composée d'un interpréteur de commandes (shell), d'un langage de script et d'un framework de gestion de la configuration. PowerShell s'exécute sur Windows, Linux et macOS.

- Il est le successeur de l'interface en ligne de commande **MS-DOS/CMD**

- **PowerShell est un interpréteur de commandes moderne** qui comprend les meilleures fonctionnalités d'autres interpréteurs de commandes populaires. Contrairement à la plupart d'entre eux, qui acceptent et retournent uniquement du texte, PowerShell accepte et retourne des objets .NET.

- **En tant que langage de script**, PowerShell est souvent utilisé pour **automatiser** la gestion des systèmes. Il sert également à créer, à tester et à déployer des solutions, souvent dans des environnements CI/CD. Toutes les entrées et sorties sont des objets .NET. Il n'est pas nécessaire d'analyser la sortie texte pour extraire des informations de la sortie.

SOLYTI

Exercice
CMD

-Déplacement dans les différents dossiers
-Création d’un dossier dans un emplacement
-Création d’un fichier texte
-Écrire dans ce fichier texte

Les cmdlets

Les commandes dans PowerShell sont appelées cmdlets (prononcé « command-lets ») ou applets de commande. En plus des cmdlets, PowerShell vous permet d'exécuter toute commande disponible sur votre système.

Les cmdlets sont des commandes PowerShell natives, et non des exécutables autonomes. Les cmdlets sont collectées dans des modules PowerShell qui peuvent être chargés à la demande. Les cmdlets peuvent être écrites dans n'importe quel langage .NET compilé ou dans le langage de script PowerShell lui-même.

PowerShell utilise une paire verbe-nom pour nommer les cmdlets. Par exemple, la cmdlet **Get-Command** incluse dans PowerShell est utilisée pour obtenir toutes les cmdlets inscrites dans l'interface de commande. Le verbe identifie l'action effectuée par la cmdlet, et le nom identifie la ressource sur laquelle la cmdlet exécute son action

Les cmdlets

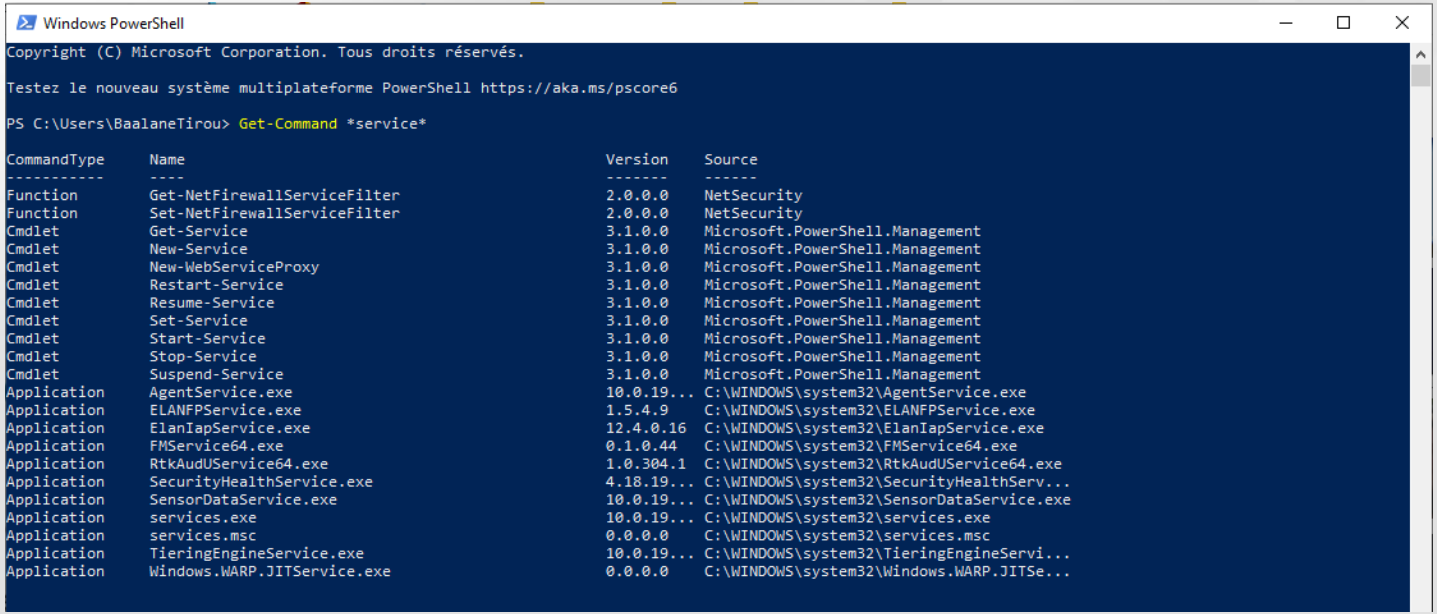
- **Une cmdlet** se compose d'un verbe et d'un nom séparés par un tiret (**verbe-nom**)
  Le « verbe » peut prendre les valeurs suivantes :

- **Get** = Obtenir quelque chose.
- **Set** = Définir quelque chose.
- **Start** = Exécuter quelque chose.
- **Stop** = Arrêter quelque chose.
- **Out** = Sortir quelque chose.
- **New** = Créer quelque chose.

SOLYTI

Les cmdlets

Exemple : ***Get-Command *services****
Tabulation => Auto-complétions



SOLYTI

Exercice

Trouvez une commande avec Get, Set, Start, Stop, et New
Et donner l'explication de la commande.

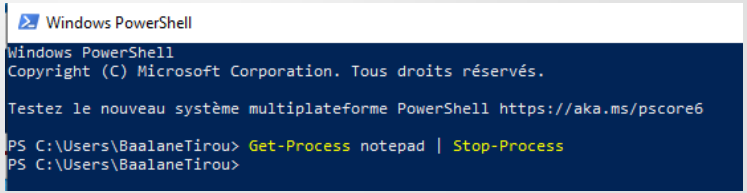
Exercice

Get-Content = Récupère le contenu d'un fichier
Set-Location = Spécifier un emplacement
Start-Service = Démarrer un service
Stop-Process = Arrêter un process
New-Item = Création d'un dossier/fichier

Pipeline

Un pipeline est une série de commandes connectées par les opérateurs de pipeline « | » Chaque opérateur de pipeline envoie les résultats de la commande précédente à la commande suivante.

La sortie de la première commande peut être envoyée pour le traitement en tant qu'entrée à la deuxième commande. Et cette sortie peut être envoyée à une autre commande. Le résultat est une chaîne de commandes ou un pipeline complexe composé d'une série de commandes simples.



La première commande utilise l'applet Get-Process de commande pour obtenir un objet représentant le processus du Bloc-notes. Il utilise un opérateur de pipeline (|) pour envoyer l'objet de processus à l'applet Stop-Process de commande, ce qui arrête le processus du Bloc-notes

SOLYTI

Exercice

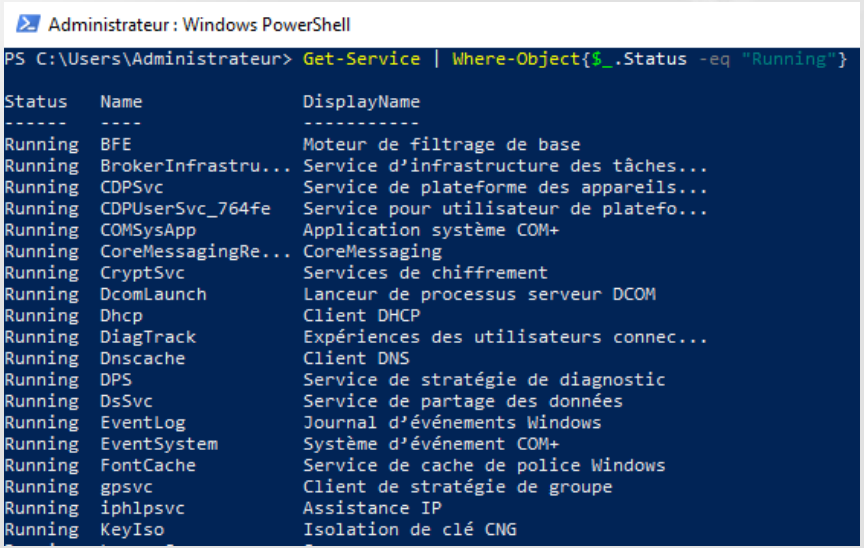
Récupérer la liste des processus en cours commençant par « wi » et rediriger le résultat Dans un fichier txt qui est situé dans un dossier du Bureau.

Récupérer la liste des Services dont le statut est en « Running »

Pipeline

- **Le pipeline** permet de marquer **une pause** pour effectuer la commande suivante
- Très pratique lorsque l'on souhaite **filtrer et affiner** le résultat

Exemple :



SOLYTI

Pipeline

PowerShell inclut une cmdlet Where-Object qui permet de tester chaque objet dans le pipeline et de le transmettre dans le pipeline uniquement s'il répond à une condition de test particulière. Les objets qui ne passent pas le test sont supprimés du pipeline.

```
PS C:\Users\BaalaneTirou> Get-Service | Where-Object {$_.Status -eq "Stopped"}

Status   Name               DisplayName
------   ----               -----------
Stopped  AarSvc_1025ac27    Agent Activation Runtime_1025ac27
Stopped  AJRouter           Service de routeur AllJoyn
Stopped  ALG                Service de la passerelle de la couc...
Stopped  AppIDSvc           Identité de l'application
Stopped  AppReadiness       Préparation des applications
Stopped  AppVClient         Microsoft App-V Client
Stopped  AssignedAccessM... Service AssignedAccessManager
Stopped  autotimesvc        Heure cellulaire
Stopped  AxInstSV           Programme d'installation ActiveX (A...
Stopped  BcastDVRUserSer... Service utilisateur de diffusion et...
Stopped  BDESVC             Service de chiffrement de lecteur B...
Stopped  BITS               Service de transfert intelligent en...
Stopped  CaptureService_... CaptureService_1025ac27
Stopped  ClipSVC            Service de licences de client (Clip...
Stopped  cloudidsvc         Service d'identité de Microsoft Cloud
Stopped  COMSysApp          Application système COM+
Stopped  ConsentUxUserSv... ConsentUX_1025ac27
Stopped  CredentialEnrol... CredentialEnrollmentManagerUserSvc_...
Stopped  CscService         Fichiers hors connexion
Stopped  defragsvc          Optimiser les lecteurs
```

SOLYTI

Pipeline

En raison de considérations liées à l'analyse, les symboles tels que <,> et = ne sont pas utilisés comme opérateurs de comparaison. Au lieu de cela, les opérateurs de comparaison sont constitués de lettres. Les opérateurs de comparaison de base sont répertoriés dans le tableau suivant

Opérateur de comparaison	Signification
-eq	Est égal à
-ne	N'est pas égal à
-lt	Est inférieur à
-le	Est inférieur ou égal à
-gt	Est supérieur à
-ge	Est supérieur ou égal à
-like	Est comme (comparaison générique pour le texte)
-notlike	N'est pas comme (comparaison générique pour le texte)
-contains	Contient
-notcontains	Ne contient pas

SOLYTI

Help

Get-Help vous indique comment utiliser les commandes une fois que vous les avez trouvées.
Quand la commande Get-Help est utilisée pour localiser des commandes, l'entrée fournie est d'abord comparée aux noms de commandes. Si l'opération ne donne rien, la recherche porte alors sur les rubriques d'aide proprement dites. Si aucune corres pondance n'est trouvée, une erreur est retournée.

```
PS C:\Users\BaalaneTirou> Get-Help -Name Get-Help

NOM
    Get-Help

RÉSUMÉ
    Displays information about PowerShell commands and concepts.


SYNTAXE
    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider | General | FAQ | Glossary | HelpFile |
    ScriptCommand | Function | Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource | Class |
    Configuration}] [-Component <System.String[]>] -Detailed [-Functionality <System.String[]>] [-Path
    <System.String>] [-Role <System.String[]>] [<CommonParameters>]
```

```
DESCRIPTION
    The `Get-Help` cmdlet displays information about PowerShell concepts and commands, including cmdlets, functions,
    Common Information Model (CIM) commands, workflows, providers, aliases, and scripts.

    To get help for a PowerShell cmdlet, type `Get-Help` followed by the cmdlet name, such as: `Get-Help Get-Process`.

    Conceptual help articles in PowerShell begin with about_ , such as about_Comparison_Operators . To see all about_
    articles, type `Get-Help about_*`. To see a particular article, type `Get-Help about_<article-name>`, such as
    `Get-Help about_Comparison_Operators`.

    To get help for a PowerShell provider, type `Get-Help` followed by the provider name. For example, to get help for
    the Certificate provider, type `Get-Help Certificate`.

    You can also type `help` or `man`, which displays one screen of text at a time. Or, `<cmdlet-name> -?`, that is
    identical to `Get-Help`, but only works for cmdlets.
```

SOLYTI

Syntaxe de l'aide

La syntaxe est principalement composée de plusieurs séries de crochets ouvrants et fermants ([]). Ceux-ci ont deux significations différentes selon la façon dont ils sont utilisés. Tout ce qui est contenu entre crochets est facultatif, sauf s'il s'agit d'une série de crochets vides []. Les crochets vides s'affichent uniquement après un type de données comme <string[]>. Cela signifie qu'un paramètre particulier peut accepter plusieurs valeurs de ce type.

```
PS C:\Users\BaalaneTirou> help Get-EventLog

NOM
    Get-EventLog

RÉSUMÉ
    Gets the events in an event log, or a list of the event logs, on the local computer or remote computers.


SYNTAXE
    Get-EventLog [-LogName] <System.String> [[-InstanceId] <System.Int64[]>] [-After <System.DateTime>]
    [-AsBaseObject] [-Before <System.DateTime>] [-ComputerName <System.String[]>] [-EntryType {Error | Information |
    FailureAudit | SuccessAudit | Warning}] [-Index <System.Int32[]>] [-Message <System.String>] [-Newest
    <System.Int32>] [-Source <System.String[]>] [-UserName <System.String[]>] [<CommonParameters>]

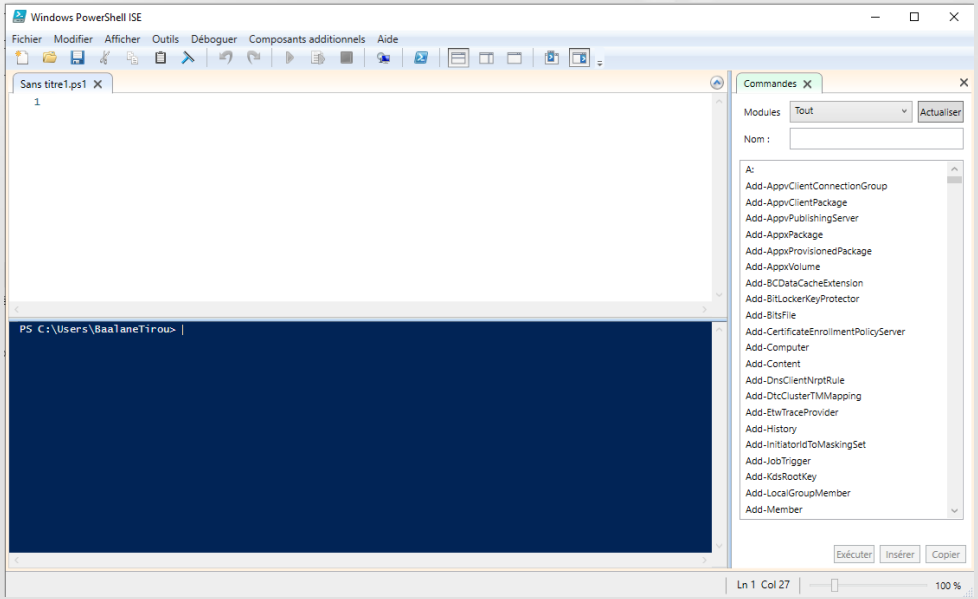
    Get-EventLog [-AsString] [-ComputerName <System.String[]>] [-List] [<CommonParameters>]
```

Le deuxième paramètre est **InstanceId**. Notez que le nom de paramètre et le type de données sont tous deux placés en totalité entre crochets angulaires. Cela signifie que le paramètre **InstanceId** est facultatif, et non obligatoire. Notez également qu'**InstanceId** est placé entre sa propre série de crochets angulaires. Comme avec le paramètre **LogName**, cela signifie que le paramètre est positionnel. Il y a une dernière série de crochets angulaires après le type de données. Cela signifie qu'il peut accepter plusieurs valeurs sous la forme d'une table ou d'une liste séparée par des virgules.

SOLYTI

Powershell ISE

Windows PowerShell Integrated Scripting Environment (ISE) est une application hôte pour Windows PowerShell. Dans ISE, vous pouvez exécuter des commandes et écrire, tester et déboguer des scripts dans une seule interface utilisateur graphique Windows. ISE fournit l'édition multiligne, la saisie semi-automatique par tabulation, la coloration syntaxique, l'exécution sélective, l'aide contextuelle et la prise en charge pour les langues s'écrivant de droite à gauche. Les éléments de menu et les raccourcis clavier sont mappés une grande partie des tâches accessibles à partir de la console Windows PowerShell. Par exemple, lorsque vous déboguez un script dans ISE, vous pouvez cliquer avec le bouton droit sur une ligne de code dans le panneau de modification pour définir un point d'arrêt.

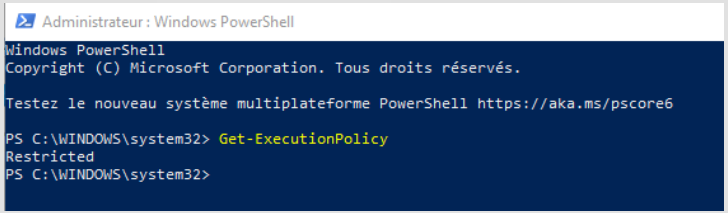


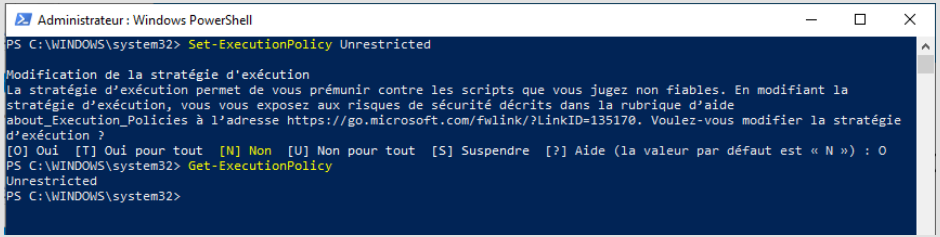
SOLYTI

L’exécution des scripts

Powershell utilise **une politique de sécurité interne** pour l’exécution de ses scripts.

- Pour connaître la politique active sur une machine : **Get-ExecutionPolicy**
- Pour modifier la politique d’exécution sur une machine : **Set-ExecutionPolicy Unrestricted**
- La politique d’exécution la plus restrictive est « **AllSigned** », et l’inverse est « **Bypass** »





SOLYTI

L’exécution des scripts

- **Restricted** : Cette valeur qui est la valeur par défaut, empêche l'exécution de scripts.

- **AllSigned** : Requiert la signature numérique par un éditeur de l'ensemble des scripts, y compris ceux que vous créez en local sur la machine.

- **RemoteSigned** : Requiert la signature numérique par un éditeur des scripts téléchargés à partir d'internet.

- **Unrestricted** : Exécute tous les scripts, mais vous invite à autoriser l'exécution des scripts non signés téléchargés via internet.

- **Bypass** : Pas de blocage, ni d'avertissements, tout sera exécuté.

- **Undefined** : Supprime la stratégie d'exécution appliquée, mais pas dans le cas où elle est définie par une stratégie de groupe.

SOLYTI

Les alias

Un alias est un autre nom ou surnom pour une applet de commande ou pour un élément de commande, tel qu'une fonction, un script, un fichier ou un fichier exécutable. Vous pouvez utiliser l'alias au lieu du nom de la commande dans n'importe quelle commande PowerShell.

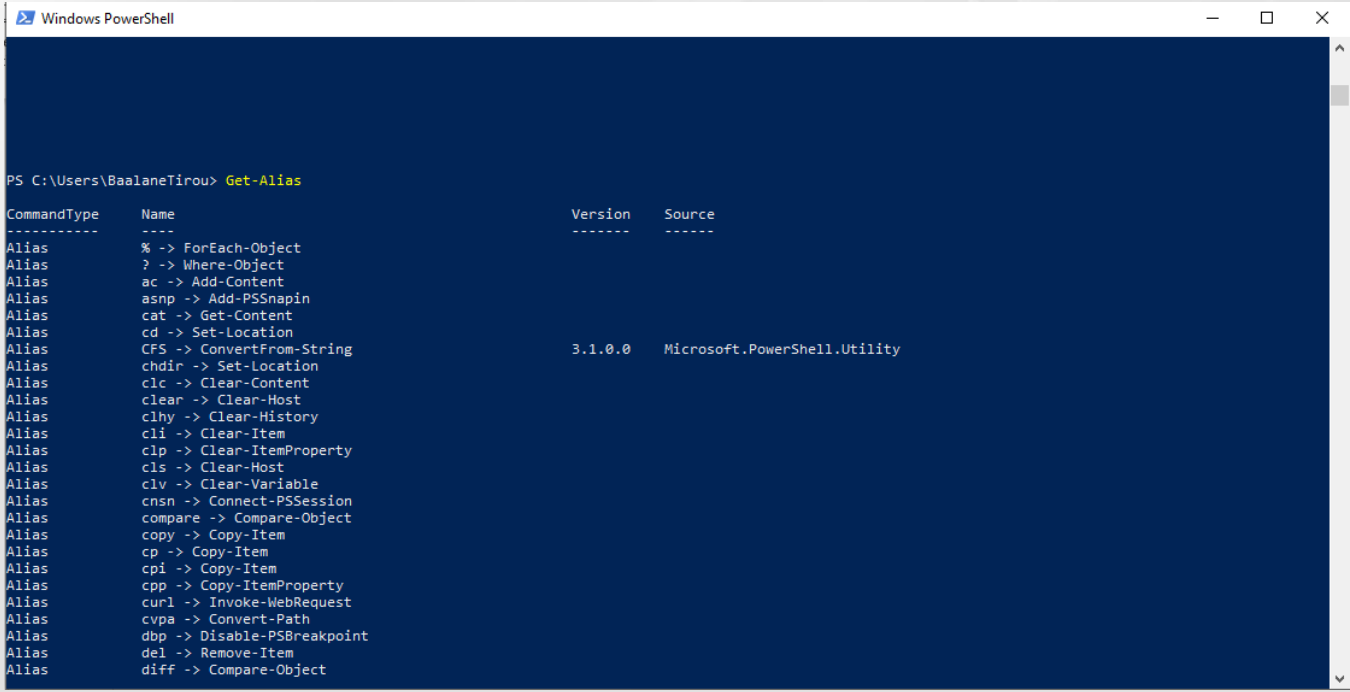
PowerShell inclut les applets de commande suivantes, qui sont conçues pour utiliser des alias :

Get-Alias - Obtient tous les alias de la session active.
New-Alias - Crée un alias.
Set-Alias - Crée ou modifie un alias.
Export-Alias - Exporte un ou plusieurs alias vers un fichier.
Import-Alias - Importe un fichier d'alias dans PowerShell.

SOLYTI

Les alias

- **Un alias** est un nom alternatif d'une commande
- **Cls** = **Clear-Host**
- Certaines commande Linux ont également leurs Alias
- **Ls** = **Get-ChildItem**



SOLYTI

Création des Alias

-Créer un alias pour ouvrir le notepad

-Créer un alias pour lister les process

-Créer un alias pour ouvrir les paramètres des cartes réseaux

-Ajouter un alias pour l’équivalant du < dir >

Les alias

Pour créer un alias, utilisez l'applet de New-Alias commande. Par exemple, la commande suivante crée l'alias info de l'applet Get-ComputerInfo de commande

```
PS C:\Users\BaalaneTirou> New-Alias -name info  -Value Get-ComputerInfo
PS C:\Users\BaalaneTirou> info
```

```
PS C:\Users\BaalaneTirou> info -Property CsManufacturer,WindowsProductName,OsArchitecture

CsManufacturer WindowsProductName OsArchitecture
-------------- ------------------ --------------
LENOVO         Windows 10 Pro     64 bits
```

SOLYTI

Objets, Propriétés et Méthodes

Get-Member vous permet de découvrir **les objets**, **les propriétés** et **les méthodes** qui sont disponibles pour les commandes. Toute commande qui produit une sortie basée sur un objet peut être redirigée vers **Get-Member**. Une propriété est une caractéristique d'un élément. Une méthode est une action qui peut être réalisée sur un élément.

```
PS C:\Users\BaalaneTirou> Get-Service -Name w32time

Status   Name               DisplayName
------   ----               -----------
Stopped  w32time            Temps Windows
```

Status, **Name** et **DisplayName** sont des exemples de propriétés, comme ceux du jeu de résultats précédent. La valeur de la propriété Status est Stopped, la valeur de la propriété Name est w32time, et la valeur de DisplayName est Windows Time.

SOLYTI

Objets, Propriétés et Méthodes

```
PS C:\Users\BaalaneTirou> Get-Service -Name w32time | Get-Member


   TypeName : System.ServiceProcess.ServiceController

Name                      MemberType    Definition
----                      ----------    ----------
Name                      AliasProperty Name = ServiceName
RequiredServices          AliasProperty RequiredServices = ServicesDependedOn
Disposed                  Event         System.EventHandler Disposed(System.Object, System.EventArgs)
Close                     Method        void Close()
Continue                  Method        void Continue()
CreateObjRef              Method        System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
Dispose                   Method        void Dispose(), void IDisposable.Dispose()
Equals                    Method        bool Equals(System.Object obj)
ExecuteCommand            Method        void ExecuteCommand(int command)
GetHashCode               Method        int GetHashCode()
```

La première ligne des résultats de l'exemple précédent contient une information très importante. TypeName indique le type d'objet retourné. Dans cet exemple, un objet System.ServiceProcess.**ServiceController** a été retourné.

SOLYTI

Objets, Propriétés et Méthodes

```
PS C:\Users\BaalaneTirou> Get-Command -ParameterType ServiceController

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Cmdlet          Get-Service                                        3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Restart-Service                                    3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Resume-Service                                     3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Set-Service                                        3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Start-Service                                      3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Stop-Service                                       3.1.0.0    Microsoft.PowerShell.Management
Cmdlet          Suspend-Service                                    3.1.0.0    Microsoft.PowerShell.Management
```

Une fois que vous connaissez le type d'objet produit par une commande, vous pouvez utiliser ces informations pour rechercher les commandes qui acceptent ce type d'objet comme entrée.

SOLYTI

Objets, Propriétés et Méthodes

Les propriétés disponibles sont plus nombreuses que celles affichées par défaut. Même si ces propriétés supplémentaires ne sont pas affichées par défaut, vous pouvez les sélectionner à partir du pipeline en redirigeant la commande vers l'applet de commande Select-Object et en utilisant le paramètre Property

```
PS C:\Users\BaalaneTirou> Get-Service -Name w32time | Select-Object -Property *


Name                  : w32time
RequiredServices      : {}
CanPauseAndContinue   : False
CanShutdown           : False
CanStop               : False
DisplayName           : Temps Windows
DependentServices     : {}
MachineName           : .
ServiceName           : w32time
ServicesDependedOn    : {}
ServiceHandle         :
Status                : Stopped
ServiceType           : Win32OwnProcess, Win32ShareProcess
StartType             : Manual
Site                  :
Container             :
```

SOLYTI

Pour la valeur du paramètre Property, vous pouvez également sélectionner certaines propriétés à l'aide d'une liste séparée par des virgules

```
PS C:\Users\BaalaneTirou> Get-Service -Name w32time | Select-Object -Property Status, Name, DisplayName, ServiceType

Status Name    DisplayName                          ServiceType
------ ----    -----------                          -----------
Stopped w32time Temps Windows Win32OwnProcess, Win32ShareProcess
```

Objets, Propriétés et Méthodes

Une méthode est une action qui peut être réalisée. Utilisez le paramètre MemberType pour affiner les résultats de Get-Member afin de n'afficher que les méthodes de Get-Service.

```
PS C:\Users\BaalaneTirou> Get-Service -Name w32time | Get-Member -MemberType Method

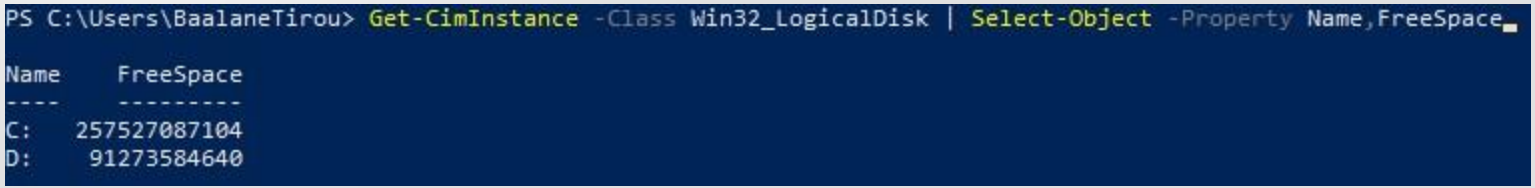

   TypeName : System.ServiceProcess.ServiceController

Name                      MemberType Definition
----                      ---------- ----------
Close                     Method     void Close()
Continue                  Method     void Continue()
CreateObjRef              Method     System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
Dispose                   Method     void Dispose(), void IDisposable.Dispose()
Equals                    Method     bool Equals(System.Object obj)
ExecuteCommand            Method     void ExecuteCommand(int command)
GetHashCode               Method     int GetHashCode()
GetLifetimeService        Method     System.Object GetLifetimeService()
GetType                   Method     type GetType()
InitializeLifetimeService Method     System.Object InitializeLifetimeService()
Pause                     Method     void Pause()
Refresh                   Method     void Refresh()
Start                     Method     void Start(), void Start(string[] args)
Stop                      Method     void Stop()
```

SOLYTI

Sélection des attributs de sortie

Vous pouvez utiliser la cmdlet Select-Object pour créer des objets PowerShell personnalisés qui contiennent des propriétés sélectionnées à partir des objets que vous utilisez pour les créer. Pour créer un nouvel objet qui inclut uniquement les propriétés Nom et FreeSpace de la classe WMI Win32_LogicalDisk



Sélection des attributs de sortie

```
PS C:\Users\BaalaneTirou> Get-CimInstance -Class Win32_LogicalDisk |
>>   Select-Object -Property Name, @{
>>     label='FreeSpace'
>>     expression={($_.FreeSpace/1GB).ToString('F2')}
>> }

Name FreeSpace
---- ---------
C:   239,84
D:   85,01
```

Avec **Select-Object** vous pouvez créer des propriétés calculées. Vous pouvez ainsi afficher FreeSpace en gigaoctets plutôt qu'en octets.

SOLYTI

Qu’est-ce que WMI ?

WMI (Windows Management Instrumentation) est une invention technologique de Microsoft, dont le but est de prendre en charge des différents environnements opérationnels de Windows.

Windows Management Instrumentation (WMI) consiste en un ensemble d’extensions du modèle de pilote Windows, qui fournissent une interface de système d’exploitation par laquelle ses composants nous donnent des informations et différents types de notifications.

WMI permet aux langages de script (tels que VBScript ou Windows PowerShell) de gérer les ordinateurs personnels et les serveurs Microsoft Windows

SOLYTI

Qu’est-ce que WMI ?

WMI (Windows Management Instrumentation) est une technologie majeure pour l’administration du système Windows, car elle expose un vaste éventail d’informations de manière uniforme. Compte tenu de tout ce que WMI permet de réaliser, la cmdlet Windows PowerShell pour l’accès aux objets WMI, **Get-CimInstance**, est l’une des plus utiles pour travailler réellement. Nous allons aborder l’utilisation des CimCmdlets pour accéder aux objets WMI, puis expliquer comment utiliser des objets WMI pour réaliser des opérations spécifique

SOLYTI

Exercice

Afficher les informations de la class qui permet de récupérer les informations de L’OS, puis récupérer les propriétés

Qu’est-ce que WMI ?

La classe Win32_OperatingSystem a beaucoup plus de propriétés que celles affichées ici. Vous pouvez utiliser l’applet de commande Get-Member pour voir toutes les propriétés

```
PS C:\Users\BaalaneTirou> Get-CimInstance -Class Win32_OperatingSystem | Get-Member -MemberType Property


   TypeName : Microsoft.Management.Infrastructure.CimInstance#root/cimv2/Win32_OperatingSystem

Name                                   MemberType Definition
----                                   ---------- ----------
BootDevice                             Property   string BootDevice {get;}
BuildNumber                            Property   string BuildNumber {get;}
BuildType                              Property   string BuildType {get;}
Caption                                Property   string Caption {get;}
CodeSet                                Property   string CodeSet {get;}
CountryCode                            Property   string CountryCode {get;}
CreationClassName                      Property   string CreationClassName {get;}
CSCreationClassName                    Property   string CSCreationClassName {get;}
CSDVersion                             Property   string CSDVersion {get;}
CSName                                 Property   string CSName {get;}
CurrentTimeZone                        Property   int16 CurrentTimeZone {get;}
DataExecutionPrevention_32BitApplications Property   bool DataExecutionPrevention_32BitApplications {get;}
DataExecutionPrevention_Available      Property   bool DataExecutionPrevention_Available {get;}
DataExecutionPrevention_Drivers        Property   bool DataExecutionPrevention_Drivers {get;}
DataExecutionPrevention_SupportPolicy  Property   byte DataExecutionPrevention_SupportPolicy {get;}
Debug                                  Property   bool Debug {get;}
Description                            Property   string Description {get;set;}
Distributed                            Property   bool Distributed {get;}
EncryptionLevel                        Property   uint32 EncryptionLevel {get;}
ForegroundApplicationBoost             Property   byte ForegroundApplicationBoost {get;set;}
FreePhysicalMemory                     Property   uint64 FreePhysicalMemory {get;}
FreeSpaceInPagingFiles                 Property   uint64 FreeSpaceInPagingFiles {get;}
FreeVirtualMemory                      Property   uint64 FreeVirtualMemory {get;}
InstallDate                            Property   CimInstance#DateTime InstallDate {get;}
LargeSystemCache                       Property   uint32 LargeSystemCache {get;}
LastBootUpTime                         Property   CimInstance#DateTime LastBootUpTime {get;}
LocalDateTime                          Property   CimInstance#DateTime LocalDateTime {get;}
Locale                                 Property   string Locale {get;}
Manufacturer                           Property   string Manufacturer {get;}
```

SOLYTI

Tri d'objets

Vous pouvez organiser des données affichées pour faciliter leur analyse en utilisant l'applet de commande Sort-Object. L'applet de commande Sort-Object prend le nom d'une ou plusieurs propriétés pour trier et retourner les données triées sur les valeurs de ces propriétés

```
NOM
    Sort-Object

RÉSUMÉ
    Sorts objects by property values.


SYNTAXE
    Sort-Object [[-Property] <System.Object[]>] [-CaseSensitive] [-Culture <System.String>] [-Descending]
    [-InputObject <System.Management.Automation.PSObject>] [-Unique] [<CommonParameters>]
```

SOLYTI

Exporter les résultats : CSV, XML…

L’applet Export-CSV de commande crée un fichier CSV des objets que vous envoyez. Chaque objet est une ligne qui inclut une liste séparée par des virgules des valeurs de propriété de l’objet. Vous pouvez utiliser l’applet Export-CSV de commande pour créer des feuilles de calcul et partager des données avec des programmes qui acceptent des fichiers CSV comme entrée

Exercice Export : Récupérer les services et afficher le nom, le nom afficher et le statut l’exporter en csv sur le Bureau.

Exercice Tri d’objet : Récupérer les éléments présent a la racine du C: et trier par date d’écriture

SOLYTI

```
PS C:\Users\BaalaneTirou> Get-Service | Select-Object Name, DisplayName, Status | Export-CSV -Path "C:\TEMP\services.csv" -Delimiter "
;" -NoTypeInformation
PS C:\Users\BaalaneTirou>
```

```
PS C:\Users\BaalaneTirou> Get-ChildItem | Sort-Object -Property Name


    Répertoire : C:\Users\BaalaneTirou


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        09/08/2022     09:04                .docker
d-----        01/07/2022     09:43                .ms-ad
-a----        18/07/2022     14:32            190 .packettracer
d-----        06/06/2022     15:06                .vscode
d-r---        31/05/2022     21:46                3D Objects
d-----        22/06/2022     11:54                Cisco Packet Tracer 8.1.1
d-r---        31/05/2022     21:46                Contacts
d-r---        11/08/2022     09:50                Desktop
d-r---        05/08/2022     14:49                Documents
d-r---        11/08/2022     10:10                Downloads
d-r---        31/05/2022     21:46                Favorites
d-r---        31/05/2022     21:46                Links
d-r---        31/05/2022     21:46                Music
d-r---        31/05/2022     22:32                OneDrive
dar--l        11/08/2022     09:40                OneDrive - SOLYTI
d-r---        31/05/2022     21:47                Pictures
d-r---        31/05/2022     21:46                Saved Games
d-r---        07/06/2022     09:36                Searches
d-r---        31/05/2022     22:34                SOLYTI
d-r---        10/06/2022     09:11                Videos
```

Les variables

Vous pouvez stocker tous les types de valeurs dans les variables PowerShell. Par exemple, stockez les résultats des commandes et stockez les éléments utilisés dans les commandes et les expressions, tels que les noms, les chemins, les paramètres et les valeurs.

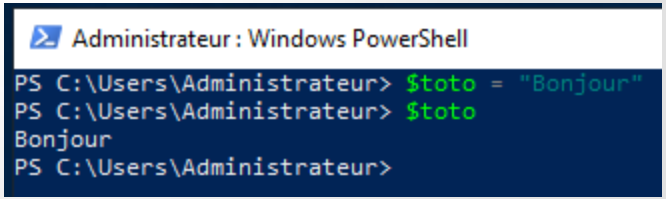
Une variable est une unité de mémoire dans laquelle les valeurs sont stockées. Dans PowerShell, les variables sont représentées par des chaînes de texte qui commencent par un signe dollar ($par exemple, $a, $process ou $test).

Les noms de variables ne respectent pas la casse et peuvent inclure des espaces et des caractères spéciaux. Toutefois, les noms de variables qui incluent des caractères spéciaux et des espaces sont difficiles à utiliser et doivent être évités.

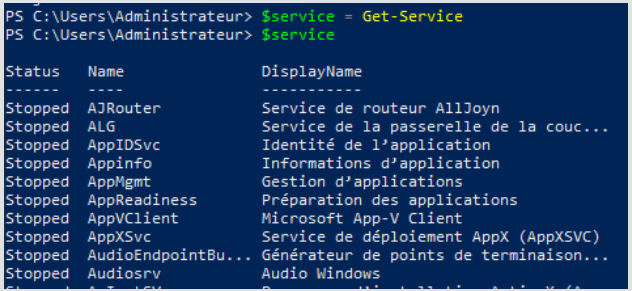
SOLYTI

Les variables

- Pour déclarer une **variable**, ajouter le préfixe **« $ »** : **$Toto**



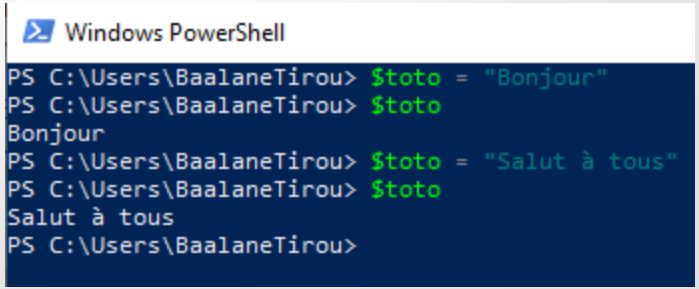
- On peut **stocker le résultat** d'une commande dans **une variable** :



SOLYTI

Les variables

Pour modifier la valeur d'une variable, affectez une nouvelle valeur à la variable.



SOLYTI

Les variables

Pour supprimer la valeur d'une variable, utilisez l'applet Clear-Variable de commande ou modifiez la valeur $null

```
PS C:\Users\BaalaneTirou> Clear-Variable -Name toto
PS C:\Users\BaalaneTirou> $toto
PS C:\Users\BaalaneTirou>
```

```
PS C:\Users\BaalaneTirou> $toto = "Salut à tous"
PS C:\Users\BaalaneTirou> $toto
Salut à tous
PS C:\Users\BaalaneTirou> $toto = $null
PS C:\Users\BaalaneTirou> $toto
PS C:\Users\BaalaneTirou>
```

SOLYTI

Les opérateurs arithmétiques

Les opérateurs arithmétiques calculent des valeurs numériques. Vous pouvez utiliser un ou plusieurs opérateurs arithmétiques pour ajouter, soustraire, multiplier et diviser des valeurs, et calculer le reste (module) d'une opération de division.

Opérateur	Description	Exemple
+	Ajoute des entiers ; Concatène	6 + 2
-	Permet de soustraire une valeur d'une autre	6 - 2
*	Multiplier les nombres ou copier des chaînes	6 * 2
/	Divise deux valeurs.	6 / 2

SOLYTI

Les opérateurs arithmétiques

PowerShell traite les opérateurs arithmétiques dans l'ordre suivant :

Priorité	Opérateur	Description
1	()	Parenthèses
2	-	Pour un nombre négatif ou un opérateur unaire
3	*, /, %	Pour la multiplication et la division
4	+, -	Pour l'ajout et la soustraction

```
PS C:\Users\BaalaneTirou> 3+6/3*4
11
PS C:\Users\BaalaneTirou> 3+6/(3*4)
3,5
PS C:\Users\BaalaneTirou> (3+6)/3*4
12
PS C:\Users\BaalaneTirou>
```

SOLYTI

Les opérateurs affectation

Les opérateurs d'affectation attribuent une ou plusieurs valeurs à une variable. Ils peuvent effectuer des opérations numériques sur les valeurs avant l'affectation.

Opérateur	Description
=	Définit la valeur d'une variable sur la valeur spécifiée.
+=	Augmente la valeur d'une variable par la valeur spécifiée, ou ajoute la valeur spécifiée à la valeur existante.
-=	Diminue la valeur d'une variable par la valeur spécifiée.
*=	Multiplie la valeur d'une variable par la valeur spécifiée, ou ajoute la valeur spécifiée à la valeur existante.
/=	Divise la valeur d'une variable par la valeur spécifiée.

SOLYTI

Exercice

Créer une variable avec à la fois les process en cours, les services, et le chiffre 7

Créer une autre variable avec le chemin du Profil windows (C:\Users\nom)
-Sans indiquer le chemin dans la variable
Ajouter dans cette même variable le chiffre 5 et la date

Les opérateurs affectation

Par conséquent, vous pouvez affecter n'importe quel type d'objet à la collection.

```
PS C:\Users\BaalaneTirou> $a = 12
PS C:\Users\BaalaneTirou> $a += 7
PS C:\Users\BaalaneTirou> $a += 15
PS C:\Users\BaalaneTirou> $a
34
```

```
PS C:\Users\BaalaneTirou> $a = Get-Service
PS C:\Users\BaalaneTirou> $a += Get-Date
PS C:\Users\BaalaneTirou> $a += 12
PS C:\Users\BaalaneTirou> $a

Status   Name               DisplayName
------   ----               -----------
Stopped  AarSvc_1025ac27    Agent Activation Runtime_1025ac27
Running  AdobeARMservice    Adobe Acrobat Update Service
Running  agent_ovpnconnect  OpenVPN Agent agent_ovpnconnect
Stopped  AJRouter           Service de routeur AllJoyn
Stopped  ALG                Service de la passerelle de la couc...
Running  AMD External Ev... AMD External Events Utility
Running  AMD Log Utility    AMD Log Utility
Stopped  AppIDSvc           Identité de l'application

DisplayHint : DateTime
Date        : 11/08/2022 00:00:00
Day         : 11
DayOfWeek   : Thursday
DayOfYear   : 223
Hour        : 16
Kind        : Local
Millisecond : 535
Minute      : 9
Month       : 8
Second      : 30
Ticks       : 637958309705353203
TimeOfDay   : 16:09:30.5353203
Year        : 2022
DateTime    : jeudi 11 août 2022 16:09:30

12
```

SOLYTI

Les opérateurs logique

Les opérateurs de comparaison dans PowerShell peuvent comparer deux valeurs ou filtrer des éléments d'une collection par rapport à une valeur d'entrée.

Opérateurs d'égalité
-eq et -ne
Lorsque le côté gauche est scalaire, -eq retourne True si le côté droit est une correspondance exacte, sinon, -eq retourne False. -ne fait l'inverse; elle retourne False lorsque les deux côtés correspondent; sinon, -ne retourne True.

```
PS C:\Users\BaalaneTirou> 2 -eq 2
True
PS C:\Users\BaalaneTirou> 2 -eq 3
False
PS C:\Users\BaalaneTirou> "abc" -ne "def"
True
PS C:\Users\BaalaneTirou> "abc" -ne "abc"
False
PS C:\Users\BaalaneTirou>
```

SOLYTI

Structure conditionnelle If, Else et ElseIf

Un bloc de structure conditionnelle commence toujours par une instruction "If" et elle sera éventuellement suivie par une ou plusieurs instructions Else/ElseIf. Il peut également y avoir une instruction "If" seule mais ce n'est pas possible pour les instructions Else et ElseIf

En français, et afin de faciliter aussi la compréhension, on peut traduire ces trois instructions :

- If = Si...
- Else = Sinon...
- ElseIf = Sinon si...

SOLYTI

Structure conditionnelle If, Else et ElseIf

La partie "condition" va permettre de tester une ou plusieurs conditions. Dans le cas où la condition se vérifie, alors le bloc de code sera exécuté. Ce bloc de code peut contenir un autre If (conditions If imbriquées), une boucle d'un autre type, ou tout autre commande : il n'y a pas de restriction à ce niveau-là

Condition If

Exercice

Si $texte est égal à « AJC" alors on affiche "Bienvenue !"

```
PS C:\Users\BaalaneTirou> $texte = "AJC"
If($texte -eq "AJC")
{
  Write-Output "Bienvenue !"
}
Bienvenue !

PS C:\Users\BaalaneTirou>
```

SOLYTI

Condition If / Else en PowerShell

Si $texte est égal à « AJC" alors on écrit "Bienvenue !" sinon on écrit « Erreur !« Afficher « Erreur ! »

```
PS C:\Users\BaalaneTirou> $texte = "Microsoft"

If($texte -eq "AJC")
{
  Write-Output "Bienvenue !"
}else{
  Write-Output "Erreur !"
}
Erreur !
```

SOLYTI

Condition If / ElseIf / Else en PowerShell

```
PS C:\Users\BaalaneTirou> $texte = "AJC"

If($texte -eq "Microsoft")
{
  Write-Output "Bienvenue !"
}elseif($texte -eq "AJC")
{
  Write-Output "Bienvenue chez AJC !"
}else{
  Write-Output "Erreur !"
}
Bienvenue chez AJC !
```

Il est important de préciser que l'instruction "Else" doit toujours être la dernière si vous désirez inclure une ou plusieurs instructions ElseIf

Si $texte est égal à « Microsoft » alors on écrit "Bienvenue !" sinon si $texte est égal à « AJC » alors on écrit « Bienvenue chez AJC !», sinon on écrit « Erreur ! »
Afficher « Bienvenue chez AJC »

SOLYTI

Double condition avec If et -or / -and

L'exemple précédent à base If/ElseIf/Else a l'avantage de permettre d'exécuter un bloc de code différent en fonction de si $texte = « AJC » ou $texte = « Microsoft ». Néanmoins, nous pourrions avoir besoin d'exécuter le même code que $texte soit égal à « AJC » ou « Microsoft »

Il suffit d'inclure deux conditions à tester dans l'instruction "If" à l'aide -or ("ou"). Ceci va permettre de tester si l'une des deux conditions est vraie. Si vous souhaitez que les deux soient vraies, vous devrez utiliser -and (et).

Si $texte est égal à « Microsoft »ou à « AJC » alors on écrit "Bienvenue !" sinon on écrit « Oups ! »
Afficher « Bienvenue chez AJC »

```
PS C:\Users\BaalaneTirou> $texte = "AJC"

If(($texte -eq "AJC") -or ($texte -eq "Microsoft"))
{
  Write-Output "Bienvenue !"
}else
{
  Write-Output "Oups !"
}
Bienvenue !

PS C:\Users\BaalaneTirou>
```

$pair est égal à 2 et $impair =3
Si $pair est égal à 1 et $impair =3 alors on écrit « C'est vrai sinon on écrit « C'est faux»

```
PS C:\Users\BaalaneTirou> $pair = 2
$impair = 3

If(($pair -eq 1) -and ($impair -eq 3))
{
  Write-Output "C'est vrai !"}
  Else{ Write-Output "C'est Faux !"
  }

C'est Faux !

PS C:\Users\BaalaneTirou> |
```

Les boucles For

Lorsque l'on utilise une boucle for, on indique une valeur de départ ,une valeur cible dans la condition de répétition et on incrémente la valeur à chaque tour de boucle.

```
PS C:\Users\BaalaneTirou> $valeur = 0..10
For($i=0;$i -lt 10;$i++)
{
   Write-Output "La valeur est $($valeur[$i])"
}
La valeur est 0
La valeur est 1
La valeur est 2
La valeur est 3
La valeur est 4
La valeur est 5
La valeur est 6
La valeur est 7
La valeur est 8
La valeur est 9
```

Affecter la valeur "0..10" à une variable sert à créer un tableau de valeurs, de 0 à 10.

On initialise la valeur de $i à 0 qui sera inférieur à 10. Puis on incrémente la valeur de $i

SOLYTI

Les fonctions

Une fonction est une liste d'instructions PowerShell qui ont un nom que vous attribuez. Lorsque vous exécutez une fonction, vous tapez le nom de la fonction. Les instructions de la liste s'exécutent comme si vous les avez tapées à l'invite de commandes.

Exercice : Créer une fonction pour m'afficher la version du Powershell

```
PS C:\Users\BaalaneTirou> function Get-PSVersion {
>>     $PSVersionTable.PSVersion
>> }
PS C:\Users\BaalaneTirou> Get-PSVersion

Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      19041  1682


PS C:\Users\BaalaneTirou>
```

SOLYTI

Les modules

Un module est un package qui contient des membres PowerShell, tels que des applets de commande, des fournisseurs, des fonctions, des workflows, des variables et des alias. Les membres de ce package peuvent être implémentés dans un script PowerShell, une DLL compilée ou une combinaison des deux. Ces fichiers sont généralement regroupés dans un seul répertoire

- **Les modules** PowerShell encapsulent un **ensemble de fonctionnalités** communes. Par exemple, il existe des modules PowerShell spécifiques aux fournisseurs qui gèrent différents services de cloud (Azure, Vmware)
- Ça permet **d'ajouter des fonctionnalités supplémentaires**

- Les commandes pour gérer les modules :

Get-Module –ListAvailable : Lister les modules disponibles sur la machines
Find-Module : Rechercher un module

- En utilisant la commande **$env:PSModulePath** nous affichons l'emplacement des modules

SOLYTI

Les modules

- **Les modules** PowerShell encapsulent un **ensemble de fonctionnalités** communes. Par exemple, il existe des modules PowerShell spécifiques aux fournisseurs qui gèrent différents services de cloud (Azure, Vmware)
- Ça permet **d'ajouter des fonctionnalités supplémentaires**

- Les commandes pour gérer les modules :

Get-Module –ListAvailable : Lister les modules disponibles sur la machines

Find-Module : Rechercher un module

- En utilisant la commande **$env:PSModulePath** nous affichons l'emplacement des modules

```
PS C:\Users\Administrateur> $env:PSModulePath
C:\Users\Administrateur\Documents\WindowsPowerShell\Modules;C:\Program Files\WindowsPowerShell\Modules;C:\Windows\system32\Windows
PowerShell\v1.0\Modules
PS C:\Users\Administrateur>
```

SOLYTI

Gestion des lecteurs Windows PowerShell

Un lecteur Windows PowerShell est un emplacement de magasin de données auquel vous pouvez accéder au même titre qu'un lecteur du système de fichiers dans Windows PowerShell. Les fournisseurs Windows PowerShell créent pour vous certains lecteurs, comme les lecteurs du système de fichiers (y compris C: et D:), les lecteurs de Registre (HKCU: et HKLM:) et le lecteur de certificat (Cert:). Vous pouvez également créer vos propres lecteurs Windows PowerShell. Ces lecteurs sont très utiles, mais ils ne sont disponibles que dans Windows PowerShell. Vous ne pouvez pas y accéder à l'aide d'autres outils Windows, tels que l'Explorateur de fichiers ou Cmd.exe.

Les commandes Windows PowerShell associées aux lecteurs Windows PowerShell comportent le nom PSDrive dans leur intitulé. Pour obtenir la liste des lecteurs Windows PowerShell dans votre session Windows PowerShell, utilisez l'applet de commande Get-PSDrive.

SOLYTI

Gestion des lecteurs Windows PowerShell

Exercice

Créer un nouveau lecteur dédié a Hyper-V
Avec comme emplacement racine l'emplacement de stockages des VM

```
PS C:\Users\Baalane> New-PSDrive -Name Hyper-V -PSProvider FileSystem -Root "C:\Users\Public\Documents"

Name           Used (GB)     Free (GB) Provider      Root
----           ---------     --------- --------      ----
Hyper-V             0,00         33,28 FileSystem    C:\Users\Public\Documents
```

SOLYTI